

Esame Programmazione di Sistema / API Programming

Data: 2021-07-05

THEORETICAL AND PROGRAMMING QUESTIONS

Si implementi in linguaggio Rust una struttura generica `Buffer<T>` che modelli una struttura dati condivisa tra due thread concorrenti, uno produttore e uno consumatore. La struttura deve consentire al produttore di inserire valori e notificare la terminazione della produzione, mentre il consumatore può richiedere valori in modalità FIFO, rispettando le seguenti regole:

- **Metodo `next(...)`:** Il thread produttore utilizza questo metodo per aggiungere un nuovo valore al buffer. Se è stata invocata una chiamata a `terminate()` o `fail(...)`, il metodo deve fallire lanciando un'eccezione e il buffer deve rimanere inalterato.
- **Metodo `terminate()`:** Il thread produttore utilizza questo metodo per notificare che non saranno disponibili ulteriori valori. Dopo questa chiamata, il buffer non accetta più nuovi valori. Se non ci sono valori nel buffer, il metodo `consume()` deve restituire `None`.
- **Metodo `fail(...)`:** Il thread produttore utilizza questo metodo per notificare un errore e indicare che non saranno disponibili ulteriori valori. Dopo questa chiamata, il buffer non accetta più nuovi valori. Se non ci sono valori nel buffer, il metodo `consume()` deve restituire un errore.
- **Metodo `consume()`:** Il thread consumatore utilizza questo metodo per prelevare un valore dal buffer in modalità FIFO. Se non ci sono valori disponibili, il metodo si blocca in attesa di nuovi valori o di una condizione di terminazione, senza consumare cicli di CPU. Se è stato invocato `terminate()` e non ci sono valori, restituisce `None`. Se è stato invocato `fail(...)` e non ci sono valori, rilancia l'eccezione specificata.

```
1 impl<T: Send> Buffer<T> {
2     pub fn new() -> Self;
3
4     // Metodi relativi alla produzione di valori
5     pub fn next(&self, value: T);
6     pub fn terminate(&self);
7     pub fn fail(&self, error: Box<dyn Any + Send>);
8
9     // Metodo relativo al consumo di valori
10    pub fn consume(&self) -> Result<Option<T>, Box<dyn Any + Send>>;
11 }
```

ANSWERS

Analisi del Problema

Il problema richiede l'implementazione di un buffer concorrente (*unbounded*) che consenta la comunicazione FIFO tra thread produttori e consumatori. La particolarità di questo buffer è la sua macchina a stati, che può transitare da uno stato aperto (**Open**) a uno stato chiuso con successo (**Terminated**) o chiuso con errore (**Failed**). In entrambi i casi di chiusura, il buffer smette di accettare nuovi valori. Poiché l'oggetto `Buffer<T>` viene condiviso per reference (`&self`), è necessario utilizzare primitive di sincronizzazione quali `Mutex` e `Condvar` incapsulate all'interno della struttura, allocandole nell'heap tramite `Arc`.

Implementazione in Rust

Di seguito la proposta di soluzione completa e commentata. Si definisce un enumerativo `State` per modellare con precisione la macchina a stati descritta nei requisiti.

```
1 use std::sync::{Arc, Mutex, Condvar};
2 use std::any::Any;
3 use std::collections::VecDeque;
4
5 // Rappresenta lo stato interno del Buffer
6 enum State<T> {
7     Open(VecDeque<T>),
8     Terminated(VecDeque<T>),
9     Failed(VecDeque<T>, Option<Box<dyn Any + Send>>),
10 }
11
12 pub struct Buffer<T> {
13     data: Arc<(Mutex<State<T>>, Condvar)>,
14 }
15
16 // Permette ai vari thread di avere una propria copia (reference-counted)
17 // del Buffer
18 impl<T> Clone for Buffer<T> {
19     fn clone(&self) -> Self {
20         Buffer {
21             data: Arc::clone(&self.data),
22         }
23     }
24 }
25
26 impl<T: Send> Buffer<T> {
27     pub fn new() -> Self {
28         Buffer {
29             data: Arc::new((Mutex::new(State::Open(VecDeque::new())),
30                             Condvar::new())),
31         }
32     }
33
34     pub fn next(&self, value: T) {
35         let (lock, cvar) = &*self.data;
36         let mut state = lock.lock().unwrap();
37         match &mut *state {
38             State::Open(q) => {
39                 q.push_back(value);
40             }
41         }
42     }
43 }
```

```

38         // Notifichiamo il thread consumatore in attesa (se
presente)
39         cvar.notify_one();
40     }
41     State::Terminated(_) | State::Failed(_, _) => {
42         // Se chiuso, fallisce lanciando un'eccezione (panic in
Rust)
43         panic!("Cannot add elements to a terminated or failed
buffer");
44     }
45 }
46 }
47
48 pub fn terminate(&self) {
49     let (lock, cvar) = &*self.data;
50     let mut state = lock.lock().unwrap();
51     if let State::Open(q) = &mut *state {
52         // Estrae la coda corrente lasciando una vuota al suo posto
(tramite Default),
53         // in modo da poter cambiare il variant in Terminated senza
allocazioni inutili
54         let queue = std::mem::take(q);
55         *state = State::Terminated(queue);
56         // Sveglia tutti i thread in attesa, affinche' vedano il
cambio di stato
57         cvar.notify_all();
58     }
59 }
60
61 pub fn fail(&self, error: Box<dyn Any + Send>) {
62     let (lock, cvar) = &*self.data;
63     let mut state = lock.lock().unwrap();
64     if let State::Open(q) = &mut *state {
65         let queue = std::mem::take(q);
66         *state = State::Failed(queue, Some(error));
67         cvar.notify_all();
68     }
69 }
70
71 pub fn consume(&self) -> Result<Option<T>, Box<dyn Any + Send>> {
72     let (lock, cvar) = &*self.data;
73     let mut state = lock.lock().unwrap();
74     loop {
75         match &mut *state {
76             State::Open(q) => {
77                 if let Some(v) = q.pop_front() {
78                     return Ok(Some(v));
79                 }
80                 // Nessun valore e buffer aperto: attende senza
consumare cicli di CPU
81                 state = cvar.wait(state).unwrap();
82             }
83             State::Terminated(q) => {
84                 if let Some(v) = q.pop_front() {
85                     return Ok(Some(v));
86                 } else {
87                     // Buffer chiuso e vuoto
88                     return Ok(None);

```

```

89         }
90     }
91     State::Failed(q, err) => {
92         if let Some(v) = q.pop_front() {
93             return Ok(Some(v));
94         } else if let Some(e) = err.take() {
95             // Consuma l'errore registrato e lo restituisce
96             return Err(e);
97         } else {
98             // L'errore e' gia' stato consumato in
precedenza
99             panic!("Buffer already failed and error consumed
100         }
101     }
102 }
103 }
104 }
105 }

```